

프로젝트 결과 보고서

프로젝트명 : 이미지 분류와 RAG 기반 가구 추천 챗봇

01 프로젝트 요약

한 줄 소개

- 사용자가 업로드한 가구 이미지를 분류하고, 예측된 카테고리나 질문을 바탕으로 관련 문서를 검색해 추천 답변을 생성하는 LLM/RAG 프로토타입입니다.

프로젝트 목표

- 목표: 딥러닝 이미지 분류와 RAG 검색, LLM 답변 생성이 어떻게 연결되는지 보여준다.

핵심 기능

- 이미지 업로드 기반 가구 카테고리 분류
- 예측 카테고리나 사용자 질문을 함께 활용한 RAG 검색
- 검색된 근거 문서를 바탕으로 추천 답변 생성
- 외부 LLM 연결 실패 시 RAG 템플릿 답변으로 대체

대표 결과

- 예측 클래스: 수납장, 의자, 냉장고, 테이블, TV
- 추천 기준: 카테고리, 사용 목적, 배치 조건, 사용자 질문
- 결과 화면: 예측 카테고리, CONFIDENCE, 추천 답변, RAG 근거 문서

사용 기술

- PYTHON, STREAMLIT
- TENSORFLOW/KERAS, MOBILENETV2
- TF-IDF, COSINE SIMILARITY
- GEMINI API, GROQ API, OLLAMA, FALLBACK TEMPLATE

02 문제 정의와 시스템 구조

문제 정의

- 가구 추천은 텍스트 조건뿐 아니라 이미지, 공간, 용도, 배치 조건의 영향을 받는다.
- 사용자의 이미지와 질문을 함께 활용하면 더 구체적인 추천 흐름을 만들 수 있다.
- 검색 근거를 보여주는 RAG 구조로 답변 신뢰도를 높이하고자 했다.

전체 흐름

이미지 업로드 -> 전처리 ->
MOBILENETV2 분류 -> 카테고리 추출
-> RAG 문서 검색 -> LLM 답변 생성 ->
STREAMLIT 출력

기술 구성

- 이미지 분류: MOBILENETV2 기반 전이학습
- 입력 크기: 160 X 160 RGB
- 검색: TF-IDF + COSINE SIMILARITY
- 답변: GEMINI/GROQ/OLLAMA/FALLBACK

03 RAG 검색과 답변 생성 구현

RAG 검색

- `TFIDFVECTORIZER`로 지식 문서를 벡터화
- 사용자 질문을 벡터로 변환
- `COSINE\_SIMILARITY`로 질문과 문서의 유사도 계산
- 이미지 예측 카테고리과 같은 문서에는 BONUS 적용
- TOP-K 문서를 답변 근거로 사용

LLM 답변

- 검색된 문서를 CONTEXT로 구성
- GEMINI API 연결 시 외부 LLM 답변 생성
- GROQ API를 보조 연결로 사용
- OLLAMA 로컬 모델도 후보로 두었음

FALLBACK

- 외부 LLM 호출 실패 시 앱이 멈추지 않음
- 검색된 RAG 문서 기반 템플릿 답변 제공
- 사용자 화면에는 답변 SOURCE와 오류 상태를 표시

```
def retrieve(self, query, category=None, top_k=3):
    query_vector = self.vectorizer.transform([query])
    scores = cosine_similarity(query_vector, self.matrix)[0]

    ranked = []
    for idx, score in enumerate(scores):
        doc = self.documents[idx]
        category_bonus = 0.2 if category and doc.category == category else 0
        ranked.append((score + category_bonus, doc))

    ranked.sort(key=lambda item: item[0], reverse=True)
    return [doc for _, doc in ranked[:top_k]]
```

04 시연 화면과 이미지 전처리

시연 화면

가구 이미지 업로드

생성된 이미지 5.png1.8MB+

이미지 분석하기

침대, 의자, 수납장, 테이블, tv처럼 가구가 잘 보이는 이미지를 올려주세요.



업로드 이미지

분류 가능 카테고리

수납장 almirah

의자 chair

냉장고 fridge

테이블 table

TV tv

예측 결과와 추천 상담

현재 이미지 기준 예측 결과

의자 (chair)

모델이 이 이미지를 의자 카테고리로 분류했습니다. 예측 확률은 99.66%입니다.

좌석

책상

작업공간

의자

수납장

TV

테이블

냉장고

0102030405060708090100

예측 확률(%)

추천 질문

배치 추천해줘

추천 답변 받기

Gemini API / gemini-2.5-flash

추천 답변

이미지 분류 결과, 의자로 확인됩니다.

테이블이나 책상 등 함께 사용할 가구와 높이 및 색상을 맞춰 배치하시면 좋습니다.

좁은 공간에서는 팔걸이가 없는 의자가 이동과 수납에 편리하니 참고해 주세요.

결과 해석

- 이미지 분류 결과가 RAG 검색 조건으로 이어진다.
- 사용자는 예측 카테고리과 추천 답변을 한 화면에서 확인할 수 있다.
- RAG 근거 문서를 펼쳐 답변의 출처를 확인할 수 있다.

▼ 검색된 RAG 근거 문서 보기

의자 배치 팁 [chair]

의자는 테이블, 책상, 화장대 등 함께 사용하는 가구와 높이와 색상을 맞추는 것이 중요하다. 좁은 공간에서는 팔걸이가 없는 의자를 선택하면 이동과 수납이 편하다. 포인트 컬러 의자는 공간에 생동감을 줄 수 있다.

프로젝트 정보

RAG 데이터

현재 프로토타입은 TF-IDF 기반 검색과 템플릿 답변으로 RAG 흐름을 확인하는 구조입니다.

	title	category	content
1	수납장 추천 가이드 [almirah]	almirah	수납장은 보관 목적과 설치 공간을 먼저 고려해야 한다. 책, 생활용품, 의류, 주방용품 등 어떤 물건을
2	수납장 배치 팁 [almirah]	almirah	수납장은 동선을 방해하지 않는 벽면에 배치하는 것이 좋다. 높은 수납장은 안정감을 위해 벽에 붙여
3	의자 추천 가이드 [chair]	chair	의자는 사용 목적에 따라 선택 기준이 달라진다. 책상 의자는 적절한 높이와 허리 지지감이 중요하고,
4	의자 배치 팁 [chair]	chair	의자는 테이블, 책상, 화장대 등 함께 사용하는 가구와 높이와 색상을 맞추는 것이 중요하다. 좁은 공
5	냉장고 추천 가이드 [fridge]	fridge	냉장고는 용량, 문 형태, 설치 공간, 소비전력을 함께 고려해야 한다. 1인 가구나 작은 주방에는 소형 모
6	냉장고 배치 팁 [fridge]	fridge	냉장고는 문을 열 수 있는 여유 공간과 열 배출 공간이 필요하다. 벽에 너무 붙이면 냉각 효율이 떨어질
7	테이블 추천 가이드 [table]	table	테이블은 식사, 작업, 수납 등 사용 목적이 다양하므로 용도를 먼저 정해야 한다. 원형 테이블은 부드러운
8	테이블 배치 팁 [table]	table	테이블 주변에는 의자가 움직일 수 있는 여유 공간이 필요하다. 거실 테이블은 소파와 높이를 맞추는
9	TV 추천 가이드 [tv]	tv	TV는 화면 크기, 시청 거리, 해상도, 설치 방식을 고려해야 한다. 거실에서는 시청 거리와 벽면 크기에
10	TV 배치 팁 [tv]	tv	TV는 뒷 반사가 심하지 않은 위치에 배치하는 것이 좋다. 소파나 의자에서 바라보는 높이를 맞추면 시

05 트러블 슈팅과 개선 방향

트러블 슈팅

2. OLLAMA 연동 실패

문제: 로컬 OLLAMA 서버와 LLM 연동이 반복 실패

원인: 로컬 서버 실행 상태, API 연결, 메모리 환경 문제

해결: GEMINI API 연동, GROQ 보조 연결, 최종 FALLBACK 답변 구조 추가

```
def generate_chat_answer_with_meta(
    self,
    question,
    retrieved_docs,
    category_ko=None,
    category_en=None,
    confidence=None,
):
    fallback_answer = self._template_answer(
        question=question,
        retrieved_docs=retrieved_docs,
        category_ko=category_ko,
        category_en=category_en,
        confidence=confidence,
    )
    prompt = self._build_ollama_prompt(
        question=question,
        retrieved_docs=retrieved_docs,
        category_ko=category_ko,
        category_en=category_en,
        confidence=confidence,
    )

    last_error = None

    if self.gemini_api_key:
        for model in self.gemini_models:
            try:
                answer = self._call_gemini(model=model, prompt=prompt)
                if answer:
                    return {
                        "answer": answer,
                        "source": "gemini",
                    }
```

```
                "model": model,
                "error": None,
            }
        except Exception as exc:
            last_error = (
                f"{model} @ Gemini API: "
                f"{exc.__class__.__name__}: {exc}"
            )

    if self.groq_api_key:
        for model in self.groq_models:
            try:
                answer = self._call_groq(model=model, prompt=prompt)
                if answer:
                    return {
                        "answer": answer,
                        "source": "groq",
                        "model": model,
                        "error": None,
                    }
            except Exception as exc:
                last_error = (
                    f"{model} @ {self.groq_url}: "
                    f"{exc.__class__.__name__}: {exc}"
                )

    ollama_available, ollama_error = self._check_ollama()
    if ollama_available:
        for model in self.model_candidates:
            try:
                answer = self._call_ollama(model=model, prompt=prompt)
                if answer:
                    return {
                        "answer": answer,
```

```
                    "model": model,
                    "error": None,
                }
            except Exception as exc:
                last_error = (
                    f"{model} @ {self.ollama_url}: "
                    f"{exc.__class__.__name__}: {exc}"
                )

    elif not self.gemini_api_key and not self.groq_api_key:
        last_error = ollama_error

    return {
        "answer": fallback_answer,
        "source": "fallback",
        "model": "RAG template",
        "error": last_error,
    }
```

05 트러블 슈팅과 개선 방향

트러블 슈팅

2. 이미지 비율에 따른 예측 흔들림

문제: 업로드 이미지 비율과 배경에 따라 예측 확률이 흔들림

해결: DIRECT, SQUARE, CROP, MIRROR 전처리 결과를 평균 내는 방식 적용

```
class FurnitureImagePredictor:
    def __init__(self, model_path=None, class_names_path=None):
        base_dir = Path(__file__).resolve().parent
        self.model_path = self._find_existing_path(
            [
                Path(model_path) if model_path else None,
                base_dir / "furniture_transfer_160_compat.keras",
                base_dir / "furniture_transfer_160_best_compat.keras",
                base_dir / "furniture_transfer_160.keras",
                base_dir / "furniture_transfer_160_best.keras",
                base_dir / "models" / "furniture_transfer_160_compat.keras",
                base_dir / "models" / "furniture_transfer_160_best_compat.keras",
                base_dir / "models" / "furniture_transfer_160.keras",
                base_dir / "models" / "furniture_transfer_160_best.keras",
            ],
            "model",
        )
        self.class_names_path = self._find_optional_path(
            [
                Path(class_names_path) if class_names_path else None,
                base_dir / "class_names.json",
                base_dir / "models" / "class_names.json",
            ]
        )
        self.class_labels, self.image_size = self._load_class_labels()
        self.model = load_model(self.model_path)
```

개선 방향

- LANGCHAIN 또는 VECTOR STORE로 RAG 구조 고도화
- 실제 상품 DB와 가격, 재고, 사용자 취향 정보 연결
- OPENAI API 등 안정적인 LLM PROVIDER 연동
- PLUSHOME(프로젝트)과 연결해 이미지 기반 상품 추천 기능으로 확장

소감

이 프로젝트를 통해 이미지 분류 결과를 검색 조건으로 연결하고, 외부 LLM 장애 상황에서도 사용자가 답변을 받을 수 있는 RAG 기반 FALLBACK 구조를 구현했습니다.

감사합니다

THANK YOU!